

2026-05-08-phase5-end-user-materials

Phase 5 — End-User Materials Uplift Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Close all documentation gaps per USER_MANUAL_EXPANSION_PLAN.md, produce installers for Linux/macOS/Windows/Docker, and update the website to match reality.

Architecture: Three sequential work streams — documentation first (content is foundation), installers second (packaging the product), website last (reflects final docs content). Each stream has independent parallel sub-tasks.

Tech Stack: Go 1.24, dpkg-deb/rpmbuild (Linux packaging), Homebrew (macOS), NSIS/WiX (Windows), Docker, vanilla HTML/CSS/JS (website)

Design spec: docs/superpowers/specs/2026-05-08-phase5-end-user-materials-design.md

File Structure

Documentation modifications

File	Action	Responsibility
docs/user_manual/README.md	Edit — add sections in Parts III/IV/V/X	Provider deep-dives, advanced features, developer guides, troubleshooting
docs/user_manual/SUMMARY.md	Edit — sync topic index	Cross-reference integrity
docs/user_manual/INDEX.md	Edit — sync keyword index	Cross-reference integrity
docs/user_manual/tutorials/Tutorial_*.md	Edit — minor stale-reference fixes	Cross-reference integrity
docs/COMPLETE_*.md	Edit — fix stale config keys if any	Cross-reference integrity

Installer new files

File	Responsibility
scripts/ install.sh	Platform-agnostic Linux/macOS installer (detects OS/arch, downloads package)
scripts/ install.ps1	Windows PowerShell installer
packaging/debian/ control	Debian package metadata
packaging/debian/ postinst	Post-installation script
packaging/debian/ prerm	Pre-removal script
packaging/debian/ helixcode.service	systemd service unit
packaging/rpm/ helixcode.spec	RPM spec file
packaging/rpm/ helixcode.service	systemd service unit (same as debian variant or separate)
packaging/ homebrew/ helixcode.rb	Homebrew formula
packaging/ windows/ installer.nsi	NSIS installer script
packaging/docker/ Dockerfile	Minimal production Dockerfile
Makefile	Edit — add installers target

Website modifications

File	Action
github_pages_website/ docs/index.html	Edit — stats, links, feature list
github_pages_website/ docs/manual/index.html	Edit — sync manual
github_pages_website/ docs/courses/	Edit — if stale
github_pages_website/ docs/mobile/	Edit — if stale

Task DOC-1: Provider Deep-Dive Guides

Files: - Modify: docs/user_manual/README.md (Part III — add sections after existing §6-7)



Step 1: Read existing Part III structure

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
grep -n "^### " docs/user_manual/README.md | grep -E "(6\\.|7\\.|
    Anthropic|Gemini|Bedrock|Azure|XAI|OpenRouter)"
```



Step 2: Add Anthropic/Claude extended thinking and prompt caching guide

Add a new subsection after §6.1 Anthropic Claude in docs/user_manual/README.md:

```
#### 6.1.1 Extended Thinking
Enable extended thinking in config.yaml:
```yaml
llm:
 providers:
 anthropic:
 api_key: "${ANTHROPIC_API_KEY}"
 extended_thinking: true
 thinking_budget: 16000 # tokens allocated to reasoning
```

When enabled, Claude produces a thinking block before the response. HelixCode surfaces this in the UI as an expandable reasoning trace. Use for complex debugging, architectural decisions, and multi-step planning.

### 6.1.2 Prompt Caching

Cache frequently used context (system prompts, tool definitions, document excerpts):

```
llm:
 providers:
 anthropic:
 prompt_caching: true
 cache_ttl: 300 # seconds
```

Cache hit reduces cost by ~90% and latency by ~50%. Breakpoints: system prompts (automatically cached), tool definitions (automatically cached), and document blocks (manual cache\_control points).

- [ ] \*\*Step 3: Add Google Gemini multimodal and 2M context guide\*\*

Add a new subsection after §6.2 Google Gemini:

### 6.2.1 Multimodal Inputs

Gemini accepts images, video, and audio inline:

```
llm:
 providers:
 gemini:
 api_key: "${GEMINI_API_KEY}"
 multimodal: true
```

Usage: attach image files with `@image.png` in prompts. Gemini 2.5 Pro handles up to 2M tokens — enough for full codebase analysis.

### 6.2.2 Flash Models for Speed

```
llm:
 providers:
 gemini:
 model: "gemini-2.5-flash-preview-05-06"
 parameters:
 temperature: 0.7
```

Flash models offer 4x faster inference at lower cost, suitable for code generation and refactoring.

- [ ] **\*\*Step 4: Add AWS Bedrock and Azure OpenAI enterprise setup guide\*\***

Add after §6.4 AWS Bedrock and §6.5 Azure OpenAI:

### 6.4.1 AWS IAM Authentication

```
llm:
 providers:
 bedrock:
 region: "us-east-1"
 credentials:
 access_key_id: "${AWS_ACCESS_KEY_ID}"
 secret_access_key: "${AWS_SECRET_ACCESS_KEY}"
 session_token: "${AWS_SESSION_TOKEN}" # optional, for
 temporary creds
 models:
 - "anthropic.claude-sonnet-4-20250505"
 - "amazon.titan-text-premier-v1:0"
```

IAM policy must grant `bedrock:InvokeModel` and `bedrock:InvokeModelWithResponseStream`. Cross-region inference requires `bedrock:ListFoundationModels`.

### 6.5.1 Azure Entra ID Authentication

```
llm:
 providers:
 azure:
 endpoint: "https://<resource>.openai.azure.com"
 deployment_id: "gpt-4o"
 api_version: "2025-05-01-preview"
 auth:
 type: "entra_id"
 client_id: "${AZURE_CLIENT_ID}"
 client_secret: "${AZURE_CLIENT_SECRET}"
 tenant_id: "${AZURE_TENANT_ID}"
```

Requires Azure AI Service role assignment. Managed identity also supported via `auth.type: "managed_identity"`.

- [ ] **\*\*Step 5: Add free provider guides (XAI/Grok, OpenRouter, Qwen)\*\***

Add after §7 (Free & Open Source Providers):

### 7.7 XAI (Grok) Free Tier

```
llm:
 providers:
 xai:
 api_key: "${XAI_API_KEY}"
 model: "grok-3-fast-beta" # free tier
 parameters:
 temperature: 0.7
```

Free tier: 100 requests/hour, 16K context. Upgrade for 1M context and priority access.

### 7.8 OpenRouter Free Models

```
llm:
 providers:
 openrouter:
 api_key: "${OPENROUTER_API_KEY}"
 free_tier: true # automatically selects free models
```

Routes to models like `mistralai/mixtral-8x22b-instruct` (free), `google/gemma-2-27b-it` (free). Rate-limited to 20 req/min on free tier.

### 7.9 Qwen Free Quota

```
llm:
 providers:
 qwen:
```

```
api_key: "${QWEN_API_KEY}"
model: "qwen-max"
```

Free: 2,000 requests/day. Paid: 10,000 req/min. Supports tool calling and streaming.

- [ ] **\*\*Step 6: Verify sections exist in README\*\***

```
```bash
```

```
grep -c "Extended Thinking\|Prompt Caching\|Multimodal Inputs\|IAM Authent
```

Expected: 7 matches minimum.



Step 7: Commit

```
git add docs/user_manual/README.md
git commit -m "docs: add provider deep-dive guides (Anthropic,
    Gemini, AWS/Azure, free tiers)"
```

Task DOC-2: Advanced Feature Guides

Files: - Modify: docs/user_manual/README.md (Parts IV-V — add sections after tool/workflow chapters)



Step 1: Read existing Part IV structure

```
grep -n "^### " docs/user_manual/README.md | grep -E "(8\.|9\.|
    10\.|11\.|12\.|13\.)"
```



Step 2: Add memory system guide

Add after existing Part V (Advanced Workflows) or as a new chapter:

```
### 16. Memory System
```

```
#### 16.1 Available Backends
```

Backend	Persistence	Scaling	Use Case
In-Memory	None (ephemeral)	Single process	Development, testing
Filesystem	Disk (JSON)	Single node	Small projects
Redis	RAM + RDB/AOF	Distributed	High-speed caching
Memcached	RAM (volatile)	Distributed	Session cache
ChromaDB	Disk (vector)	Distributed	Semantic search
Qdrant	Disk (vector)	Distributed	Production RAG
Weaviate	Disk (vector + hybrid)	Distributed	Enterprise RAG
Cognee	Disk (graph + vector)	Distributed	AI-native memory

```
#### 16.2 Configuration
```

```
```yaml
```

```
memory:
 backend: "chromadb" # or redis, filesystem, qdrant, weaviate, cognee
 connection:
 host: "localhost"
 port: 8000
 tls: false
 persistence:
 path: "/var/lib/helixcode/memory/"
 sync_interval: 60 # seconds
 vector:
 dimensions: 1536 # match embedding model output
 distance: "cosine" # cosine, euclidean, dot
```

## 16.3 Choosing the Right Backend

- **Development:** Start with `filesystem` — zero dependencies, data survives restart
- **Team:** Use `redis` — fast, shared across team members
- **Semantic search:** Use `chromadb` or `qdrant` — vector similarity for context retrieval
- **Enterprise:** Use `weaviate` or `cognee` — hybrid search, multi-tenancy, audit trails

- [ ] **\*\*Step 3: Add tool sandboxing guide\*\***

Add to §9 Shell Execution:

## 9.5 Sandbox Configuration

```
shell:
 sandbox:
 enabled: true
 allowed_commands:
 - "go"
 - "python"
 - "node"
 - "npm"
 - "git"
 - "docker"
 - "make"
 - "curl"
 - "ls"
 - "cat"
 - "grep"
 - "find"
 blocked_commands:
 - "rm -rf /"
 - "dd"
 - "mkfs"
 - ":(){ :|:& };:" # fork bomb
 timeout: 300 # seconds per command
 max_output_size: 10485760 # 10MB
```

Commands not in `allowed_commands` require interactive confirmation. Commands matching `blocked_commands` are rejected outright. Set `sandbox.enabled: false` only in trusted, isolated environments.

- [ ] **\*\*Step 4: Add web tools enablement guide\*\***

Add to §11 Web Tools:

#### 11.4 Search Provider Configuration

```
web:
 search:
 provider: "tavily" # or google, bing, duckduckgo
 api_key: "${TAVILY_API_KEY}"
 max_results: 8
 safe_search: true
 fetch:
 user_agent: "helix_code/2.0"
 timeout: 30
 max_size: 5242880 # 5MB
```

Supported providers: Tavily (recommended — AI-optimized), Google Custom Search, Bing Web Search, DuckDuckGo (no API key needed, rate-limited).

- [ ] **\*\*Step 5: Verify sections\*\***

```
```bash
grep -c "Memory System\|Sandbox Configuration\|Search Provider" docs/user_
```

Expected: 3 matches minimum.



Step 6: Commit

```
git add docs/user_manual/README.md
git commit -m "docs: add advanced feature guides (memory,
               sandbox, web tools)"
```

Task DOC-3: Developer Guides

Files: - Create: `docs/user_manual/tutorials/Tutorial_9_Custom_Provider.md` - Create: `docs/user_manual/tutorials/Tutorial_10_Adding_a_Tool.md`



Step 1: Create Tutorial 9 — Custom Provider

Write `docs/user_manual/tutorials/Tutorial_9_Custom_Provider.md`:

Tutorial 9: Creating a Custom LLM Provider

****Duration****: 30 minutes | ****Level****: Advanced

Overview

Extend HelixCode with your own LLM provider by implementing the `Provider`` interface.

Prerequisites

- Go 1.24+
- Running HelixCode development environment
- Access to your LLM API

Step 1: Provider Scaffold

Create ``internal/llm/providers/myprovider/myprovider.go``:

```
```go
package myprovider

import (
 "context"
 "dev.helix.code/llm"
)

type MyProvider struct {
 apiKey string
 endpoint string
 model string
}
```

Implement the `llm.Provider` interface: - `Name() string` — return "myprovider" - `Generate(ctx context.Context, req *llm.Request) (*llm.Response, error)` - `GenerateStream(ctx context.Context, req *llm.Request) (<-chan *llm.Response, error)`

## Step 2: Configuration

Register in `config/config.yaml`:

```
llm:
 providers:
 myprovider:
 type: myprovider
 endpoint: "https://api.myprovider.com/v1"
 api_key: "${MYPROVIDER_API_KEY}"
 enabled: true
```

## Step 3: Testing

```
func TestMyProvider_Generate(t *testing.T) {
 p := &MyProvider{apiKey: os.Getenv("MYPROVIDER_API_KEY")}
 resp, err := p.Generate(context.Background(), &llm.Request{
 Messages: []llm.Message{{Role: "user", Content:
 "Hello"}}},
 })
 assert.NoError(t, err)
 assert.NotEmpty(t, resp.Content)
}
```

## Step 4: Registration

In `internal/llm/providers/registry.go`:

```
import "dev.helix.code/llm/providers/myprovider"

func init() {
 Register("myprovider", func(cfg Config) (llm.Provider,
 error) {
 return &myprovider.MyProvider{
 apiKey: cfg.APIKey,
 endpoint: cfg.Endpoint,
 model: cfg.Model,
 }, nil
 })
}
```

Build: `go build ./...` — should succeed.

- [ ] **\*\*Step 2: Create Tutorial 10 — Adding a Tool via MCP SDK\*\***

Write ``docs/user_manual/tutorials/Tutorial_10_Adding_a_Tool.md``:

````markdown`

`# Tutorial 10: Adding a New Tool via MCP SDK`

`**Duration**: 30 minutes | **Level**: Advanced`

`## Overview`

Create a custom tool that integrates with HelixCode's MCP server.

`## Prerequisites`

- HelixCode running with MCP enabled
- Go 1.24+
- Understanding of JSON-RPC

`## Step 1: Define Tool Schema`

Create ``internal/tools/mycalculator/tool.go``:

```
```go
package mycalculator

import (
 "context"
 "encoding/json"
 "fmt"
)

type CalculatorArgs struct {
 A float64 `json:"a"`
 B float64 `json:"b"`
}

func (t *CalculatorTool) Name() string { return "calculator" }
func (t *CalculatorTool) Description() string {
 return "Performs arithmetic operations on two numbers"
}

func (t *CalculatorTool) Schema() json.RawMessage {
 return json.RawMessage(`{
 "type": "object",
 "properties": {
 "a": {"type": "number"},
 "b": {"type": "number"}
 },
 "required": ["a", "b"]
 }`)
}
```

## Step 2: Implement Execute

```
func (t *CalculatorTool) Execute(ctx context.Context, args
 json.RawMessage) (string, error) {
 var parsed CalculatorArgs
 if err := json.Unmarshal(args, &parsed); err != nil {
 return "", fmt.Errorf("invalid args: %w", err)
 }
 result := parsed.A + parsed.B
 return fmt.Sprintf("%f", result), nil
}
```

## Step 3: Register with MCP

In `internal/mcp/registry.go`:

```
import "dev.helix.code/tools/mycalculator"

func init() {
```

```
 RegisterTool(&mycalculator.CalculatorTool{})
}
```

Test: Start server, connect via WebSocket, send {"jsonrpc": "2.0", "method": "tools/call", "params": {"name": "calculator", "arguments": {"a": 5, "b": 3}}}.  
Expected response: {"result": "8.000000"}.

## Step 4: Tool Confirmation (Optional)

Implement Confirmable interface for dangerous tools:

```
func (t *CalculatorTool) RequiresConfirmation() bool {
 return false // calculator is safe
}
```

- [ ] \*\*Step 3: Update SUMMARY.md to include new tutorials\*\*

```
```bash
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
```

Add to docs/user_manual/SUMMARY.md:

```
| Tutorial 9: Custom Provider | 30 min | Advanced |
| Tutorial 10: Adding a Tool | 30 min | Advanced |
```



Step 4: Commit

```
git add docs/user_manual/tutorials/Tutorial_9_Custom_Provider.md
      docs/user_manual/tutorials/Tutorial_10_Adding_a_Tool.md
      docs/user_manual/SUMMARY.md
git commit -m "docs: add developer guides (custom provider, tool
via MCP SDK)"
```

Task DOC-4: Troubleshooting Guide

Files: - Modify: docs/user_manual/README.md (Part X — expand Troubleshooting section)



Step 1: Read existing troubleshooting section

```
grep -n "^### 3[0-9]\." docs/user_manual/README.md
```



Step 2: Add SSH worker troubleshooting

Add to §30 Troubleshooting Guide:

30.1 SSH Worker Issues

Symptom	Cause	Fix
"connection refused"	Worker SSH not running	`systemctl start sshd`
"permission denied (publickey)"	Key not authorized	`ssh-copy-id user@host`
"host key verification failed"	Host key changed	`ssh-keygen -R worker@host`
"worker timed out after 30s"	Network latency	Increase `workers.health_timeout`
"worker rejected task"	Capability mismatch	Check `workers.capabilities`

30.2 LLM Connection Issues

Symptom	Cause	Fix
"401 Unauthorized"	Invalid API key	Check `HELIX_*_API_KEY` env var
"429 Too Many Requests"	Rate limited	Enable rate limiter: `llm.rate_limit`
"503 Service Unavailable"	Provider outage	Enable fallback: `llm.selector`
"model not found"	Wrong model name	Run `helix models` to list available
"context window exceeded"	Too many tokens	Enable compression: `llm.compress`

30.3 Rate Limit Handling

```yaml

llm:

rate\_limit:

enabled: true

strategy: "token\_bucket" # token\_bucket, sliding\_window, backoff

requests\_per\_minute: 60

burst: 10

backoff\_base: 2 # exponential backoff multiplier

backoff\_max: 120 # maximum wait in seconds

When rate limited, HelixCode automatically queues requests and retries with exponential backoff. Configure different limits per provider.

- [ ] \*\*Step 3: Commit\*\*

```bash

git add docs/user_manual/README.md

git commit -m "docs: add troubleshooting guide (SSH, LLM, rate limits)"

Task DOC-5: Cross-Reference Sync

Files: - Modify: docs/user_manual/SUMMARY.md - Modify: docs/user_manual/INDEX.md - Verify: docs/COMPLETE_*.md



Step 1: Walk SUMMARY.md against README.md sections

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
grep -n "^#### " docs/user_manual/README.md > /tmp/readme_sections.txt
```

```
grep -n "|.*|" docs/user_manual/SUMMARY.md > /tmp/
summary_entries.txt
diff <(cut -d' ' -f2- /tmp/readme_sections.txt | sort) <(cut
-d'|' -f2 /tmp/summary_entries.txt | sed 's/^ *//' |
sort) || true
```

Fix any mismatches in SUMMARY.md.



Step 2: Walk INDEX.md against README.md

```
grep -oP '(?<=^# ).*' docs/user_manual/README.md | while read
section; do
    if ! grep -qi "$section" docs/user_manual/INDEX.md; then
        echo "MISSING in INDEX: $section"
    fi
done
```

Add any missing index entries.



Step 3: Verify COMPLETE_*.md config keys match actual config

```
grep -rh "^[a-z]" docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md |
head -20
# Compare against actual config/config.yaml
diff <(grep -rh "^[a-z]" config/config.yaml | sort) <(grep -rh
"^[a-z]" docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md |
sort) || true
```

Fix any stale keys.



Step 4: Check Tutorial 1-8 for stale feature names

```
grep -rn "HelixCode v1\.\.0\|old-feature-name\|deprecated" docs/
user_manual/tutorials/
```

Replace any stale references.



Step 5: Commit

```
git add docs/user_manual/SUMMARY.md docs/user_manual/INDEX.md
docs/COMPLETE_CONFIGURATION_DOCUMENTATION.md docs/
user_manual/tutorials/
git commit -m "docs: cross-reference sync (SUMMARY, INDEX,
COMPLETE docs, tutorials)"
```

Task INST-1: Linux Packages (.deb)

Files: - Create: packaging/debian/control - Create: packaging/debian/postinst - Create: packaging/debian/prerm - Create: packaging/debian/helixcode.service - Modify: Makefile (add deb target)



Step 1: Create debian control file

Write helix_code/packaging/debian/control:

```
Source: helixcode
Section: devel
Priority: optional
Maintainer: Helix Development <dev@helix.code>
Build-Depends: debhelper-compat (= 13), golang-go (>= 1.24)
Standards-Version: 4.6.2
Homepage: https://helixcode.dev

Package: helixcode
Architecture: amd64 arm64
Depends: ${shlibs:Depends}, ${misc:Depends}, ca-certificates
Description: Distributed AI Development Platform
 HelixCode is an enterprise-grade distributed AI development platform
 that enables intelligent task division, work preservation, and
 multi-provider LLM integration through a unified CLI, REST API,
 and Terminal UI.
```



Step 2: Create systemd service unit

Write helix_code/packaging/debian/helixcode.service:

```
[Unit]
Description=HelixCode AI Development Server
After=network.target postgresql.service redis.service
Wants=postgresql.service redis.service

[Service]
Type=simple
User=helixcode
Group=helixcode
ExecStart=/usr/bin/helixcode server
Restart=on-failure
RestartSec=10
EnvironmentFile=-/etc/helixcode/helixcode.env
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```



Step 3: Create postinst script

Write helix_code/packaging/debian/postinst:

```
#!/bin/sh
set -e

case "$1" in
    configure)
        # Create helixcode user if not exists
        if ! getent passwd helixcode >/dev/null 2>&1; then
            adduser --system --group --home /var/lib/helixcode
            helixcode
        fi
        # Create data directory
        mkdir -p /var/lib/helixcode /etc/helixcode
        chown -R helixcode:helixcode /var/lib/helixcode
        # Create default config if missing
        if [ ! -f /etc/helixcode/config.yaml ]; then
            cp /usr/share/helixcode/config.yaml.example /etc/
            helixcode/config.yaml
        fi
        # Enable and start service
        systemctl daemon-reload
        systemctl enable helixcode.service
        systemctl start helixcode.service || true
        ;;
esac
```



Step 4: Create prerm script

Write helix_code/packaging/debian/prerm:

```
#!/bin/sh
set -e

case "$1" in
    remove|purge)
        systemctl stop helixcode.service || true
        systemctl disable helixcode.service || true
        ;;
esac
```



Step 5: Add Makefile target

Add to helix_code/Makefile:

```
.PHONY: deb
deb: build
    mkdir -p packaging/debian/helixcode/DEBIAN
```



```

mkdir -p packaging/debian/helixcode/usr/bin
mkdir -p packaging/debian/helixcode/etc/helixcode
mkdir -p packaging/debian/helixcode/lib/systemd/system
mkdir -p packaging/debian/helixcode/usr/share/doc/helixcode
cp bin/helixcode packaging/debian/helixcode/usr/bin/
cp packaging/debian/control packaging/debian/helixcode/
  DEBIAN/
cp packaging/debian/postinst packaging/debian/helixcode/
  DEBIAN/
cp packaging/debian/prerm packaging/debian/helixcode/DEBIAN/
cp packaging/debian/helixcode.service packaging/debian/
  helixcode/lib/systemd/system/
cp config/config.yaml packaging/debian/helixcode/etc/
  helixcode/config.yaml.example
cp README.md packaging/debian/helixcode/usr/share/doc/
  helixcode/
chmod 755 packaging/debian/helixcode/DEBIAN/postinst
chmod 755 packaging/debian/helixcode/DEBIAN/prerm
dpkg-deb --build packaging/debian/helixcode
mv packaging/debian/helixcode.deb packaging/helixcode_$
  (VERSION)_amd64.deb
@echo "Debian package: packaging/helixcode_$
  (VERSION)_amd64.deb"

```

.PHONY: installers

installers: deb rpm homebrew windows

```
@echo "All installers built in packaging/"
```



Step 6: Build and verify

```

cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
make deb
ls -la packaging/helixcode_*.deb
dpkg-deb --info packaging/helixcode_*.deb | grep -E "Package|
  Version|Architecture"

```



Step 7: Commit

```

git add packaging/debian/ Makefile
git commit -m "feat: add Debian packaging (.deb)"

```

Task INST-2: Linux Packages (.rpm)

Files: - Create: packaging/rpm/helixcode.spec - Create: packaging/rpm/helixcode.service - Modify: Makefile (add rpm target)



Step 1: Create RPM spec file

Write `helix_code/packaging/rpm/helixcode.spec`:

```
Name: helixcode
Version: 3.0.0
Release: 1%{?dist}
Summary: Distributed AI Development Platform
License: Proprietary
URL: https://helixcode.dev
Source0: helixcode-%{version}.tar.gz
BuildRequires: golang >= 1.24
Requires: ca-certificates

%description
HelixCode is an enterprise-grade distributed AI development platform
that enables intelligent task division, work preservation, and
multi-provider LLM integration.

%prep
%setup -q

%build
make build

%install
mkdir -p %{buildroot}%{_bindir}
mkdir -p %{buildroot}%{_sysconfdir}/helixcode
mkdir -p %{buildroot}%{_unitdir}
mkdir -p %{buildroot}%{_sharedstatedir}/helixcode
install -m 755 bin/helixcode %{buildroot}%{_bindir}/helixcode
install -m 644 config/config.yaml %{buildroot}%{_sysconfdir}/helixcode/con
install -m 644 packaging/rpm/helixcode.service %{buildroot}%{_unitdir}/hel

%pre
getent group helixcode >/dev/null || groupadd -r helixcode
getent passwd helixcode >/dev/null || useradd -r -g helixcode -d /var/lib/
exit 0

%post
systemctl daemon-reload
systemctl enable helixcode.service || true
systemctl start helixcode.service || true

%preun
systemctl stop helixcode.service || true
systemctl disable helixcode.service || true

%files
%{_bindir}/helixcode
%{_sysconfdir}/helixcode/config.yaml.example
%{_unitdir}/helixcode.service
%dir %{_sharedstatedir}/helixcode
%doc README.md
```

```
%changelog
* Thu May 08 2026 Helix Development <dev@helix.code> - 3.0.0
- Initial RPM release
```



Step 2: Create RPM systemd service

Write `helix_code/packaging/rpm/helixcode.service`:

```
[Unit]
Description=HelixCode AI Development Server
After=network.target
Wants=network.target

[Service]
Type=simple
User=helixcode
Group=helixcode
ExecStart=/usr/bin/helixcode server
Restart=on-failure
RestartSec=10
EnvironmentFile=-/etc/helixcode/helixcode.env
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```



Step 3: Add Makefile rpm target

Add to `helix_code/Makefile`:

```
.PHONY: rpm
rpm: build
    mkdir -p packaging/rpmbuild/{BUILD,RPMS,SOURCES,SPECS,SRPMS}
    cp packaging/rpm/helixcode.spec packaging/rpmbuild/SPECS/
    tar czf packaging/rpmbuild/SOURCES/helixcode-$(VERSION).tar.gz \
        --transform="s/^/helixcode-$(VERSION)\//" \
        bin/helixcode config/config.yaml README.md
    rpmbuild --define "_topdir $(PWD)/packaging/rpmbuild" \
        -bb packaging/rpmbuild/SPECS/helixcode.spec
    cp packaging/rpmbuild/RPMS/x86_64/helixcode-*.rpm packaging/
    @echo "RPM package: packaging/helixcode-$(VERSION)-1.x86_64.rpm"
```



Step 4: Build and verify

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
make rpm
```

```
ls -la packaging/helixcode-*.rpm
rpm -qip packaging/helixcode-*.rpm | grep -E "Name|Version|
Architecture"
```



Step 5: Commit

```
git add packaging/rpm/ Makefile
git commit -m "feat: add RPM packaging (.rpm)"
```

Task INST-3: macOS Homebrew Formula

Files: - Create: packaging/homebrew/helixcode.rb - Modify: Makefile (add homebrew target)



Step 1: Create Homebrew formula

Write helix_code/packaging/homebrew/helixcode.rb:

```
class Helixcode < Formula
  desc "Distributed AI Development Platform"
  homepage "https://helixcode.dev"
  url "https://github.com/helixcode/helixcode/releases/download/
      v3.0.0/helixcode-3.0.0.darwin.amd64.tar.gz"
  sha256
    "0000000000000000000000000000000000000000000000000000000000000000"
    # placeholder
  license "Proprietary"

  depends_on "go" => :build

  def install
    bin.install "helixcode"
    etc.install "config.yaml" => "helixcode/config.yaml.example"
    ohai "HelixCode installed!"
    ohai "Run 'helixcode server' to start the server"
    ohai "Run 'helix code --help' for usage"
  end

  def caveats
    <<~EOS
      HelixCode requires configuration. Copy the example config:
      cp #{etc}/helixcode/config.yaml.example #{etc}/helixcode/
      config.yaml
      Then edit #{etc}/helixcode/config.yaml with your API keys.
    EOS
  end
end
```

```

service do
  run [bin/"helixcode", "server"]
  keep_alive true
  log_path var/"log/helixcode.log"
  error_log_path var/"log/helixcode-error.log"
end

test do
  assert_match "helixcode version #{version}",
    shell_output("#{bin}/helixcode version")
end
end

```



Step 2: Add Makefile target

```

.PHONY: homebrew
homebrew: build
  mkdir -p packaging/homebrew
  tar czf packaging/helixcode-$(VERSION).darwin.amd64.tar.gz \
    -C bin helixcode \
    -C ../config config.yaml
  shasum -a 256 packaging/helixcode-$(VERSION).darwin.amd64.tar.gz \
    > packaging/helixcode-$(VERSION).darwin.amd64.tar.gz.sha256
  @echo "Homebrew archive: packaging/helixcode-$(VERSION).darwin.amd64.tar.gz"
  @echo "Formula: packaging/homebrew/helixcode.rb"

```



Step 3: Commit

```

git add packaging/homebrew/ Makefile
git commit -m "feat: add macOS Homebrew formula"

```

Task INST-4: Windows Installer

Files: - Create: packaging/windows/installer.nsi - Create: scripts/install.ps1 - Modify: Makefile (add windows target)



Step 1: Create NSIS installer script

Write helix_code/packaging/windows/installer.nsi:

```

!define PRODUCT_NAME "HelixCode"
!define PRODUCT_VERSION "3.0.0"
!define PRODUCT_PUBLISHER "Helix Development"

```

```

Name "${PRODUCT_NAME} ${PRODUCT_VERSION}"
OutFile "helixcode-${PRODUCT_VERSION}-setup.exe"
InstallDir "$PROGRAMFILES64\${PRODUCT_NAME}"

Section "Install"
    SetOutPath "$INSTDIR"
    File "bin\helixcode.exe"
    File "config\config.yaml"
    CreateDirectory "$APPDATA\HelixCode"
    CreateShortcut "$SMPROGRAMS\HelixCode.lnk" "$INSTDIR\helixcode.exe"
    WriteUninstaller "$INSTDIR\uninstall.exe"
    EnVar::SetHKLM
    EnVar::AddValue "PATH" "$INSTDIR"
SectionEnd

Section "Uninstall"
    Delete "$INSTDIR\helixcode.exe"
    Delete "$INSTDIR\config.yaml"
    Delete "$INSTDIR\uninstall.exe"
    RMDir "$INSTDIR"
    Delete "$SMPROGRAMS\HelixCode.lnk"
    EnVar::DeleteValue "PATH" "$INSTDIR"
SectionEnd

```



Step 2: Create PowerShell installer

Write helix_code/scripts/install.ps1:

```

#!/usr/bin/env pwsh
<#
.SYNOPSIS
    HelixCode Windows Installer
.DESCRIPTION
    Downloads and installs HelixCode on Windows
.PARAMETER Version
    Version to install (default: latest)
.PARAMETER InstallDir
    Installation directory (default: $env:LOCALAPPDATA\HelixCode)
#>

param(
    [string]$Version = "latest",
    [string]$InstallDir = "$env:LOCALAPPDATA\HelixCode"
)

$ErrorActionPreference = "Stop"

Write-Host "Installing HelixCode $Version..." -ForegroundColor
    Green

```

```
# Determine architecture
$arch = if ([Environment]::Is64BitOperatingSystem) { "amd64" }
        else { "386" }

# Download URL
$url = "https://github.com/helixcode/helixcode/releases/download/
        v$Version/helixcode-$Version.windows.$arch.zip"

# Download and extract
Write-Host "Downloading from $url..." -ForegroundColor Yellow
$zipPath = "$env:TEMP\helixcode.zip"
Invoke-WebRequest -Uri $url -OutFile $zipPath
Expand-Archive -Path $zipPath -DestinationPath $InstallDir -Force

# Add to PATH
$userPath = [Environment]::GetEnvironmentVariable("PATH", "User")
if ($userPath -notlike "$InstallDir*") {
    [Environment]::SetEnvironmentVariable("PATH", "$userPath;
        $InstallDir", "User")
    $env:PATH += ";$InstallDir"
}

Write-Host "HelixCode installed to $InstallDir" -ForegroundColor
    Green
Write-Host "Run 'helixcode version' to verify." -ForegroundColor
    Green
```



Step 3: Add Makefile target

```
.PHONY: windows
windows: build
    mkdir -p packaging/windows
    GOOS=windows GOARCH=amd64 go build -o bin/helixcode.exe ./
        cmd/server/
    zip -j packaging/helixcode-$(VERSION).windows.amd64.zip bin/
        helixcode.exe config/config.yaml
    @echo "Windows package: packaging/helixcode-$
        (VERSION).windows.amd64.zip"
```



Step 4: Commit

```
git add packaging/windows/ scripts/install.ps1 Makefile
git commit -m "feat: add Windows installer (NSIS + PowerShell)"
```

Task INST-5: Platform-Agnostic Install Script + Docker

Files: - Create: scripts/install.sh - Create: packaging/docker/Dockerfile -
Modify: README.md (add install badge/one-liner)



Step 1: Create install.sh

Write helix_code/scripts/install.sh:

```
#!/bin/sh
set -eu

REPO="helixcode/helixcode"
VERSION="${1:-latest}"
INSTALL_DIR="${HELIXCODE_DIR:-/usr/local/bin}"

detect_os() {
    case "$(uname -s)" in
        Linux)    echo "linux" ;;
        Darwin)   echo "darwin" ;;
        *)        echo "unsupported" ;;
    esac
}

detect_arch() {
    case "$(uname -m)" in
        x86_64|amd64) echo "amd64" ;;
        aarch64|arm64) echo "arm64" ;;
        *)            echo "unsupported" ;;
    esac
}

main() {
    os=$(detect_os)
    arch=$(detect_arch)

    if [ "$os" = "unsupported" ] || [ "$arch" = "unsupported" ];
    then
        echo "Unsupported platform: $(uname -s) $(uname -m)" >&2
        echo "See https://helixcode.dev/docs/install for
        alternatives" >&2
        exit 1
    fi

    echo "HelixCode Installer"
    echo "Platform: $os/$arch"
    echo ""
}
```



```

# Download latest release info
if [ "$VERSION" = "latest" ]; then
    download_url="https://github.com/$REPO/releases/latest/
    download/helixcode-$os-$arch.tar.gz"
else
    download_url="https://github.com/$REPO/releases/download/
    v$VERSION/helixcode-$VERSION.$os-$arch.tar.gz"
fi

tmpdir=$(mktemp -d)
echo "Downloading from $download_url ..."
if command -v curl >/dev/null 2>&1; then
    curl -fsSL "$download_url" -o "$tmpdir/helixcode.tar.gz"
elif command -v wget >/dev/null 2>&1; then
    wget -q "$download_url" -O "$tmpdir/helixcode.tar.gz"
else
    echo "Need curl or wget" >&2
    exit 1
fi

tar xzf "$tmpdir/helixcode.tar.gz" -C "$tmpdir"

# Install binary
mkdir -p "$INSTALL_DIR"
cp "$tmpdir/helixcode" "$INSTALL_DIR/helixcode"
chmod +x "$INSTALL_DIR/helixcode"

rm -rf "$tmpdir"

echo ""
echo "HelixCode installed to $INSTALL_DIR/helixcode"
echo "Run 'helixcode version' to verify."
echo ""
echo "Quick start:"
echo "  1. Copy config example: helixcode init-config"
echo "  2. Edit config:          \$EDITOR ~/.config/helixcode/
    config.yaml"
echo "  3. Start server:         helixcode server"
}

main "$@"

```



Step 2: Create production Dockerfile

Write helix_code/packaging/docker/Dockerfile:

```

FROM golang:1.24-alpine AS builder
WORKDIR /build

```

```

COPY . .
RUN go build -o /build/helixcode ./cmd/server/

FROM alpine:3.20
RUN apk add --no-cache ca-certificates tzdata
RUN addgroup -S helixcode && adduser -S -G helixcode helixcode

COPY --from=builder /build/helixcode /usr/local/bin/helixcode
COPY config/config.yaml /etc/helixcode/config.yaml

RUN mkdir -p /var/lib/helixcode && chown -R helixcode:helixcode /
    var/lib/helixcode

USER helixcode
EXPOSE 8080
VOLUME ["/var/lib/helixcode", "/etc/helixcode"]

ENTRYPOINT ["helixcode"]
CMD ["server"]

```



Step 3: Verify install.sh works (dry-run)

```

cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
chmod +x scripts/install.sh
bash -n scripts/install.sh
echo "Syntax OK"

```



Step 4: Commit

```

git add scripts/install.sh packaging/docker/Dockerfile
git commit -m "feat: add platform-agnostic installer script and
    production Dockerfile"

```

Task WEB-1: Website Content Accuracy Audit

Files: - Modify: github_pages_website/docs/index.html



Step 1: Audit actual provider count from code

```

cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
echo "Provider implementations:"
ls -ld internal/llm/providers/*/
echo ""
echo "Total providers: $(ls -ld internal/llm/providers/*/ | wc
    -l)"

```



Step 2: Audit actual feature count

```
echo "Tool categories:"
ls -d internal/tools/*/
echo ""
echo "Total tool packages: $(ls -d internal/tools/*/ | wc -l)"
```



Step 3: Audit Go LOC

```
find . -name "*.go" -not -path "./vendor/*" | xargs wc -l | tail
-1
```



Step 4: Update index.html stats

Replace the statistics section in `index.html`. Find and update: - Provider count (update from old number to actual) - Feature count (update from old number to actual) - Language count - Lines of code

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/Github-
Pages-Website
```



Step 5: Fix GitHub links

```
grep -n 'github.com/helixcode\|github.com/helix' docs/index.html
```

Verify each link resolves. Fix any that 404 (repo structure changed).



Step 6: Commit

```
git add github_pages_website/docs/index.html
git commit -m "website: update stats, provider count, and fix
links"
```

Task WEB-2: Manual Sync

Files: - Modify: `github_pages_website/docs/manual/index.html`



Step 1: Compare manual HTML against README.md

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode
# Extract section headings from both
grep -n "^### " helix_code/docs/user_manual/README.md | head -50
grep -n "class=\"section\"|<h[23]" github_pages_website/docs/
manual/index.html | head -50
```

Identify missing/extra sections.

☐

Step 2: Update manual HTML with any missing sections

Add any provider deep-dives, memory system, troubleshooting sections that were added to README.md but are missing from the website manual.

☐

Step 3: Check courses and mobile sub-sites

```
ls -la github_pages_website/docs/courses/  
cat github_pages_website/docs/courses/README.md 2>/dev/null |  
    head -20
```

If curricula/metadata is stale, update the relevant course data files.

☐

Step 4: Commit

```
git add github_pages_website/docs/manual/index.html  
        github_pages_website/docs/courses/  
git commit -m "website: sync manual and courses with latest docs"
```

Task WEB-3: Website Test Suite

Files: - None (run tests, fix failures)

☐

Step 1: Run local test script

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/Github-  
    Pages-Website  
./test-local.sh 2>&1 | tail -20
```

☐

Step 2: Run full website test

```
./test-website.sh 2>&1 | tail -30
```

☐

Step 3: Run performance test

```
./test-performance.sh 2>&1 | tail -10
```

☐

Step 4: Fix any test failures

If tests fail, inspect the specific test's output, fix the underlying issue in the relevant file, re-run.

☐

Step 5: Verify no 404s

```
grep -rn 'href="http' github_pages_website/docs/ | grep -v  
    'https://helixcode.dev\|https://github.com/helixcode' |  
head -20
```

Check each external link manually or via curl.

☐

Step 6: Commit any fixes

```
git add github_pages_website/  
git commit -m "website: fix test failures and broken links"
```

Final: Phase 5 Evidence and Close-Out

☐

Step 1: Verify all expansion plan items ticked

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode  
grep -c "Extended Thinking\|Prompt Caching\|Multimodal\|IAM\  
    Entra ID\|Memory System\|Sandbox\|Web Tools\|Custom  
    Provider\|MCP SDK\|SSH Worker\|LLM Connection\|Rate  
    Limit" docs/user_manual/README.md
```

Expected: 13+ matches (all expansion plan topics present).

☐

Step 2: Verify all installers exist

```
ls -la packaging/helixcode_*.deb packaging/helixcode-*.rpm  
    packaging/helixcode-*.darwin.*.tar.gz packaging/  
    helixcode-*.windows.*.zip
```

☐

Step 3: Verify install.sh works

```
bash -n scripts/install.sh
```

☐

Step 4: Update PROGRESS.md

Mark Phase 5 complete, update all task checkboxes, add commit SHAs.

☐

Step 5: Update CONTINUATION.md

Sync with Phase 5 close-out.

☐

Step 6: Final commit

```
git add docs/improvements/PROGRESS.md docs/CONTINUATION.md  
git commit -m "phase5: close out end-user materials uplift"
```